Bryce Tabangcura - Lab Week whatrever

Lab partner: haven bunter-gooley

Pre-Lab

1. Yes Using our first equation

$$y_{dark} = L\frac{(m+1/2)\lambda}{d}$$

We can choose a difference between some m. so we will choose m as 1 and 0

$$\Delta y_{dark} = L\frac{(1+1/2)\lambda}{d} - L\frac{(0+1/2)\lambda}{d}$$

$$\Delta y_{dark} = L\frac{\lambda}{d}$$

Then subsititue all of our values

$$\Delta y_{dark} = 1 \times \frac{650 \times 10^{-}9}{0.3 \times 10^{-}3}$$

This is around 2 millimeters aka $\Delta y_{dark} = 2.2 \times 10^{-}3m$

Preamble

This entire experiment was conducted completely in python. The following is the helper functions and preamble I use to make my labs quicker.

Most imports and functions will not be utilized, but I will leave them here anyway

```
import warnings
from math import e, log, pi, sqrt

import matplotlib.pyplot as plt
import numpy as np
from scipy import constants, odr
from scipy.stats import norm
from uncertainties import UFloat, ufloat, umath

warnings.filterwarnings("ignore", category=FutureWarning,
module="uncertainties")
warnings.filterwarnings(
    "ignore",
    category=FutureWarning,
```

```
    message=r"AffineScalarFunc\.__abs__\(\) is deprecated",
)
warnings.filterwarnings(
    "ignore", category=UserWarning, message="Using UFloat objects with
std_dev==0"
)


def fit_odr(x: list[UFloat], y: list[UFloat], zero_intercept: bool = False):
    x_nom, x_err = [v.n for v in x], [v.s for v in x]
    y_nom, y_err = [v.n for v in y], [v.s for v in y]
    data = odr.RealData(x_nom, y_nom, sx=x_err, sy=y_err)
    if zero_intercept:
        model, beta0 = odr.Model(lambda p, x: p[0] * x), [1]
    else:
        model, beta0 = odr.Model(lambda p, x: p[0] * x + p[1]), [1, 0]
    out = odr.ODR(data, model, beta0=beta0).run()
    slope = ufloat(out.beta[0], out.sd_beta[0])
    intercept = (
        ufloat(out.beta[1], out.sd_beta[1]) if not zero_intercept else
ufloat(0, 0)
    )
    y_pred = model.fcn(out.beta, np.array(x_nom))
    ss_res = np.sum((np.array(y_nom) - y_pred) ** 2)
    ss_tot = np.sum(
        np.array(y_nom) ** 2
        if zero_intercept
        else (np.array(y_nom) - np.mean(y_nom)) ** 2
    )
    r_squared = 1 - ss_res / ss_tot
    return slope, intercept, r_squared


def compsigma(a, b):
    return abs((a.n - b.n) / ((((a.s**2) + (b.s**2)) ** (1 / 2))))


def plot(
    x,
    y,
    fitodr=None,
    title="default title",
    xlabel="default xlabel",
    ylabel="default ylabel",
    show=True,
):
    if fitodr is None:
        fitodr = fit_odr(x, y)
```

```
    slope, intercept, r_squared = fitodr
    plt.figure()
    first = x[0].n
    last = x[-1].n

    plt.plot(
        [first, last],
        [first * slope.n + intercept.n, last * slope.n + intercept.n],
        "r-",
        label=f"Fit: $y = ({slope.n:.4g})x + ({intercept.n:.4g}),R^2 =
{r_squared:.4f}$",
    )

    plt.errorbar(
        [v.nominal_value for v in x],
        [v.nominal_value for v in y],
        xerr=[v.std_dev for v in x],
        yerr=[v.std_dev for v in y],
        capsize=5,
        linestyle="none",
        marker="o",
        markersize=3,
        label="Data",
    )

    plt.title(title)
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.legend()
    if show:
        plt.show()


def plot_residuals(
    x,
    y,
    fitodr=None,
    title="Residuals",
    xlabel="default xlabel",
    ylabel="Residual (y - fit)",
    show=True,
):
    if fitodr is None:
        fitodr = fit_odr(x, y)
    slope, intercept, r_squared = fitodr
    residuals = [yi - (slope * xi + intercept) for xi, yi in zip(x, y)]
    sigma_eff = [
        sqrt(yi.std_dev**2 + (slope.n * xi.std_dev) ** 2) for xi, yi in zip(x,
```

```
y)
    ]
    chi2 = sum((r.nominal_value**2) / (s**2) for r, s in zip(residuals,
sigma_eff))
    dof = len(y) - 1
    chi2_reduced = chi2 / dof
    zero_fit = (
        ufloat(0, 0),
        ufloat(0, 0),
        0.0,
    )  # was None — plot()'s label needs a float

    plot(
        x,
        residuals,
        fitodr=zero_fit,
        title=f"{title} ($\\chi^2_\\nu = {chi2_reduced:.3f}$)",
        xlabel=xlabel,
        ylabel=ylabel,
        show=show,
    )

    return chi2_reduced


def two_tailed_p(diff: float):
    z = abs(diff)
    return 2 * norm.sf(z)
```

Experiment 1

Aim

The goal of this experiment is to investigate the connections between wave constructive/deconstructive interference and different slits/holes

Method

We split this up into 4 parts.

First we set up the apparatus, This is a lazer pointing to a slit pointing to a screen which has a camera on the other side. We measured the distance from the screen to the slit

1. Single slit diffraction pattern. We taped a piece of tape vertically onto the screen, and measured the width of the tape, we then measured the width of both the piece of tape and the distance between two constructive peaks (not the middle) in pixels on the camera. We also taped the camera light.

2. Double slit interference pattern. We changed the slit to a double slit and recorded the results, we skipped calculations here to do later. We did the exact same thing of measuring the distance between two peaks in pixels.
3. Babinet's Principle. We did this by measuring the distance between two peaks in pixels on both the hole and the slit.
4. Measure the thickness of a piece of your hair. Using a micrometer, we measured the thickness of my hair, then we held the hair in the place where the slit should be and measured the diameter spacing of two parallel peaks.

Results

We measured the width of the tape to be 1.79 cm, we measured using a calliper with an uncertainty of 0.2mm. Our tape pixels was around 420, we choose a pixel uncertainty of 10, as on the graph increment of the software we were using has increments of 20, so we choose half of that of 10. We will use this for future pixel measurements. Our distance from the slit to the screen was measured to be around 76.5 cm. We measured in increments of half a centimetre, so half of that is our inaccuracy, aka 0.25cm

```
width_tape = ufloat(1.79, 0.02) / 100
tape_pixels = ufloat(420, 10)
distance_slit_screen = ufloat(76.5, 0.25) / 100

single_slit = ufloat(697, 10) - ufloat(120, 10)
```

Math time.

$$\Delta y_{dark} = \frac{Lm\lambda}{w}$$

We use our values in python, so that we can get proper error propagation, and get a light frequency of $(643 + / - 23)e - 09$. This is visibly close, and we can sigma test too. The values and calculations are below.

```
light_frequency = (0.04e-3 * width_tape * (single_slit / tape_pixels)) / (
    2 * distance_slit_screen
)
print(light_frequency)
print(compsigma(light_frequency,ufloat(642e-9,2e-9)))
```

```
(6.43+/-0.23)e-07
0.03893392854674969
```

We measured the distance in our double slit to be 585 pixels.

```
double_slit = ufloat(585, 10)
```

Using our sigma test, we show that our theoretical value is 26 sigma away. (talk about later)

```
# I'm remarkably confident we chose a double slit spacing of somewhere in the
range of 0.04, and we did the experiment wrong
print(2 * distance_slit_screen * light_frequency / (0.25e-3))
print(double_slit * width_tape / tape_pixels)
print(
    compsigma(
        double_slit * width_tape / tape_pixels,
        2 * distance_slit_screen * light_frequency / (0.25e-3),
    )
)
```

```
0.00393+/-0.00014
0.0249+/-0.0008
26.420135168191784
```

Back to basics. We measured for our Babinet's principle single slit size differences of 304 and 298 pixels, we can just compare the two sigma values, this will yield us a reasonably small sigma value.

```
# Compsigma
print(compsigma(ufloat(304, 10), ufloat(298, 10)))
```

```
0.4242640687119285
```

For my hair, we measured a slit difference from m = −1 and m = 1 of 273 pixels. And we can use this to get my hair width. On the micrometer we also measured a hair width of around 80 micrometers. We can compare sigma between the two

```
hair_px = ufloat(273, 10)

hair_width = (
    light_frequency * distance_slit_screen * 2 / (hair_px / tape_pixels *
width_tape)
)

print(compsigma(ufloat(80e-6, 5e-6), hair_width))
```

```
0.7284090547245792
```

Conclusions

All of our data seems to be in the right ball park/consistent with our theoretical results except our value of the double slit.

I'm somewhat confident that the slit used was the wrong slit, I hadn't payed enough attention to which slit we were using.

All of our values are within 1 sigma value of our theoretical, and the value of my hair on the micrometer and the value of my hair on the double slit are both consistent within 3 sigma.